Project Report: Week 1 Summary & Learnings

1. **Development Environment & Setup**

- **Jupyter Notebooks & Google Colab:** Got to know about such development notebooks which is a pretty interesting and easier way to learn concept in one cell and apply the code and try it out on the next cell.
- **Version Control Integration:** Github usage for uploading the weekly assignments and keeping track of learnings learned about it is used for version control and got to apply that knowledge in other projects as well.

1. **Python Fundamentals**

I already have a Basic-Intermediate knowledge in Python so it was a great revision for me:

- **Data Types & Structures:** Deep dive into built-in structures including **Strings, Lists, Tuples, Sets, and Dictionaries**, focusing on their mutability, indexing capabilities, and performance trade-offs.
- **Control Flow:** Mastery of conditional logic (if-elif-else) and loops (for, while) alongside control statements (break, continue) to handle algorithmic logic.
- **Functional Programming & Scope:** Understanding how to write reusable code blocks, handle positional/keyword arguments, and manage local vs. global variable scopes.
- **Recursion & Advanced Iteration:** Grasping the mechanics of functions calling themselves and utilizing built-in custom iterators.
- **Object-Oriented Programming (OOP):** Implementing structural blueprints using classes and objects to practice clean code principles.

1. **Core Libraries for Data Analysis & Quantitative Modeling**

The primary technical focus of Week 1 centered on three industry-standard libraries crucial for processing financial and sequential data.

1. **NumPy (Numerical Python)**

NumPy serves as the foundational package for scientific computing and matrix manipulation.

```
import numpy as np

# 1. Array Creation & Reshaping
raw_data = np.arange(12)
matrix = raw_data.reshape(3, 4)

# 2. Vector & Matrix Math
matrix_transpose = matrix.T
eigenvalues, eigenvectors = np.linalg.eig(np.eye(3))

# 3. Explicit Memory Allocation
independent_copy = matrix.copy()
```

- **Multi-dimensional Arrays (ndarray):** Learned to initialize and manipulate homogeneous tensors (1D vectors, 2D matrices, and higher-dimensional tensors).
- **Array Slicing & Reshaping:** Practiced multi-axis slicing syntax [start:stop:step] and reshaping structures (e.g., .reshape()) using both 'C' and Fortran 'F' ordering guidelines.
- **Memory Optimization (Views vs. Copies):** Discovered that standard indexing creates a *view* (sharing memory), whereas explicit allocation requires the .copy() function to safeguard original data.
- **Vector & Matrix Operations:** Implemented high-performance linear algebra operations, including matrix multiplication (@ or .dot()), transposition (.T), determinants (np.linalg.det), and calculating eigenvalues/eigenvectors (np.linalg.eig).
- **Broadcasting Rules:** Learned how NumPy handles arithmetic operations on arrays of differing shapes seamlessly.

1. **Pandas (Data Wrangling & File I/O)**

Pandas provides the structural framework necessary for cleaning and exploring tabular datasets.

```
import pandas as pd

# 1. Row/Column Extraction
subset_labels = df.loc[['A', 'B'], ['W', 'Y']]
subset_positions = df.iloc[0:3, 0:2]

# 2. Modifying the Structural Axis
df.drop('new_column', axis=1, inplace=True)

# 3. Aggregation & Missing Data
category_groups = df.groupby('category')['sales_amount'].su
m()
df['Age'].fillna(df['Age'].median(), inplace=True)
```

- **Data Structures:** Understood the distinction between a 1D Series and a 2D DataFrame (essentially multiple Series sharing a unified index).
- **Data Manipulation:** Mastered data extraction techniques using label-based .loc[] and integer position-based .iloc[] indexing, as well as dropping axes using the inplace=True parameter to modify data without generating redundant copies.
- **Aggregation & Merging:** Learned to use relational database operations like pd.merge() (for SQL-like inner/outer joins) and df.groupby() to aggregate transactional segments and calculate descriptive statistics (mean, median, standard deviation, min/max).
- **Data Preprocessing:** Implemented feature engineering tactics, including handling missing values via .fillna(), removing duplicates (.drop_duplicates()), outlier filtering through the Interquartile Range (IQR) method, and handling categorical data via One-Hot Encoding (pd.get_dummies()).

1. **Matplotlib & Seaborn (Data Visualization)**

Visual representation tools were explored to help interpret mathematical trends and historical market patterns.

```
import matplotlib.pyplot as plt
import seaborn as sns
```

```
# 1. Multi-panel Subplots
plt.subplot(2, 1, 1)
plt.plot(x, y_sin, label='Sine Wave')
plt.legend()

# 2. Feature Correlation Heatmaps
correlation_matrix = df.corr(numeric_only=True)
sns.heatmap(correlation_matrix, annot=True, cmap='coolwar
m')
```

- **Basic Charting:** Utilizing plt.plot() for lines, plt.scatter() for correlation plots, and plt.bar() / plt.hist() for frequency distributions.
- **Advanced Multi-plots:** Structuring a canvas layout with plt.subplot() to display independent data vectors simultaneously (e.g., tracking Sine vs. Cosine waves or price vs. volume).
- **Exploratory Heatmaps:** Integrating Seaborn (sns.heatmap) alongside matrix computations to visually cross-examine pairwise feature correlations within a dataset.

Plotting exercises were really fun to do :)